

Real-Time Financial Fraud Detection

Catching rare fraud with deep learning, and knowing when to ask a human

Nadir Əskərov · Akşin Fətəliyev · Habil Hüseynov · Səbuhi Nəzərov

AI Academy, National Artificial Intelligence Center, Azerbaijan · DLE-AI-202

github.com/sa6uhi/DL-final-project

APPROVE

REVIEW

BLOCK

Why it matters

Why is fraud detection hard?

1 in 29

online payments in our data
is fraud (3.5%)

Fraud is rare

Models see very few fraud
examples to learn from.

96.5%

accuracy of a useless model
that *never* flags fraud

Fraud changes

Criminals keep inventing tricks
the model has never seen.

3 choices

for every payment: approve,
block, or ask a human

Mistakes are costly

Missed fraud loses money;
false alarms annoy real
customers.

Our goal: a fast system that spots fraud **and knows when it is unsure**, so only the difficult cases go to a human analyst.

Three challenges, three ideas

1 Single payments

Looking at one payment alone misses patterns, like many purchases in a few minutes.



Transformer with history

Looks at the payment *and* at the card's last 5 payments.

2 New fraud tricks

A model trained on past fraud can miss a trick it has never seen.



Autoencoder

Learns what *normal* payments look like and flags anything unusual.

3 No confidence

A score of 0.6 does not tell us whether we can trust it.



Confidence check

Acts automatically only when sure; otherwise asks a human.

A small **gate** combines the two detectors, and everything runs as a web service with an analyst dashboard.

Why it matters

The data: real payments, split by time

590,540

real online payments (IEEE-CIS /
Vesta)

≈430

details per payment: amount,
card, email, device...

20,663

payments labelled as fraud



older payments

newer payments

1 Learn

The two detectors learn from the oldest 70% of payments.

2 Tune

The next 15% trains the gate and sets the cut-off.

3 Test

The newest 15% (88,581 payments) is kept for the end.

Like studying old exams, then sitting a new one: the models never see the future.

How do we measure success?

Accuracy can fool us

Saying “normal” for every payment is 96.5% accurate, but catches no fraud at all.

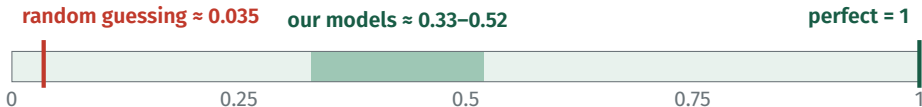
Precision

Of the payments we flag, how many are *really* fraud?

Recall

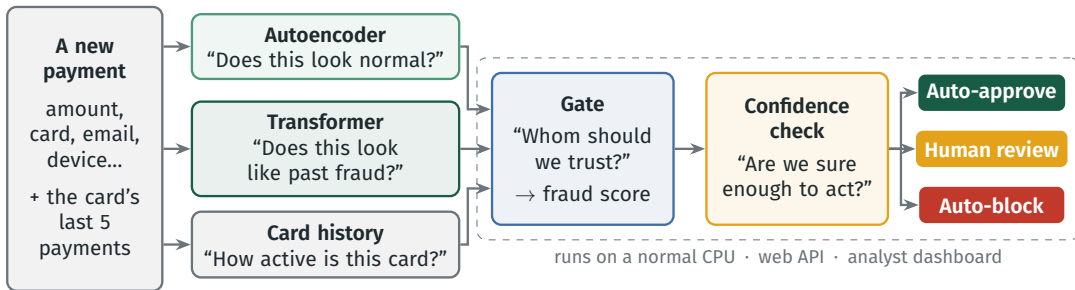
Of all the real fraud, how much did we *catch*?

PR-AUC combines precision and recall into one score from 0 to 1



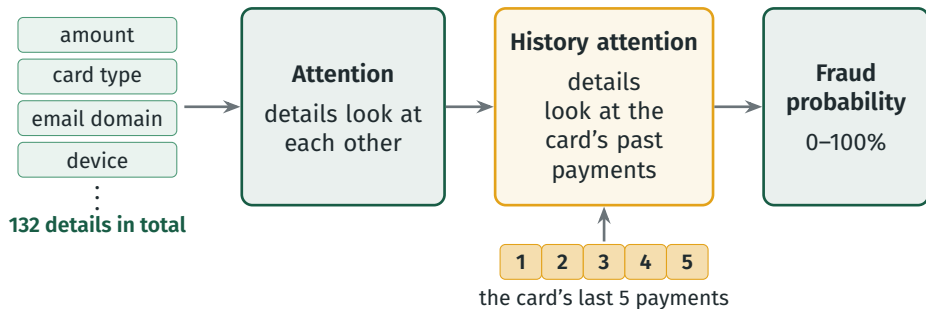
We also report how much fraud is caught when only 1 in 100 normal payments is wrongly flagged.

Our system in one picture



Think of two detectives with different skills: one knows *normal* behaviour, the other knows *past fraud*. The gate weighs their opinions, and the confidence check decides whether to act or ask a human.

The Transformer: reading a payment like a sentence



Like words in a sentence

Each detail becomes a token, the way a language model splits text into words.

Attention

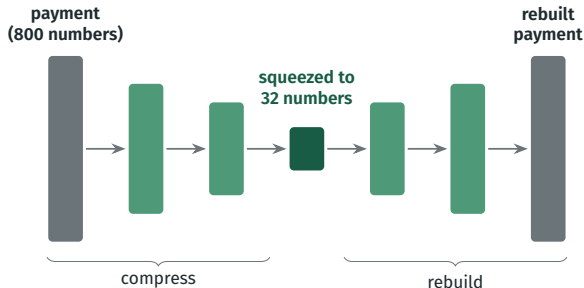
The model learns which details matter together, and which past payments matter for this one.

Learning from rare fraud

Focal loss makes rare, hard fraud examples count more. Size: 94k parameters.

How it works

The autoencoder: learning what “normal” looks like



Anomaly score = how different the rebuilt payment is from the original

Trained only on normal payments

It practises squeezing normal payments into 32 numbers and rebuilding them.

Normal → rebuilt well → low score

Unusual → rebuilt badly → high score

Why “denoising”?

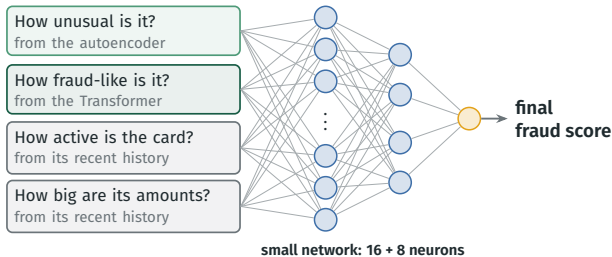
We add a little random noise during training, so it learns real patterns instead of simply copying.

Why is it useful?

It can flag new fraud tricks that nobody has labelled yet.

How it works

The gate: deciding which signal to trust



Why not a fixed 50/50 mix?

A fixed mix treats every payment the same way. A learned gate can weigh the signals differently in different situations.

No cheating

It is trained on a later time period than the two detectors, so it never sees their training data.

Tiny and fast

Only 225 parameters, and about 0.12 ms per payment.

How it works

The confidence check: act only when we are sure

1 Hold out fresh payments

44,291 payments the models never learned from.

2 Measure the mistakes

Check how wrong the fraud score can be on them.

3 Set a safety cut-off

Chosen so the right answer is covered 99% of the time.

Sure it is normal

Auto-approve

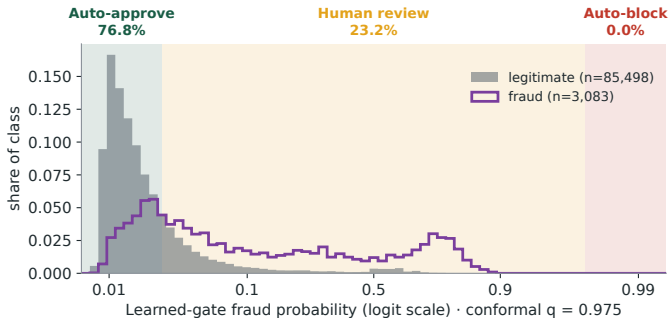
Sure it is fraud

Auto-block

Not sure

Human review

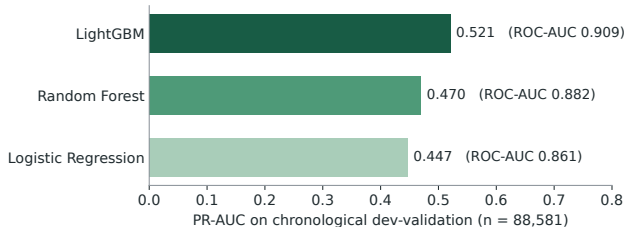
This method is called *conformal prediction*.



Most normal payments get a very low fraud score.

Results

Classic models set a strong benchmark



Measured on the tuning period (88,581 payments).

Logistic regression

A simple weighted sum of the payment details.

Random forest

200 decision trees that vote together.

LightGBM

Trees built one after another, each fixing the previous one's mistakes.

LightGBM scores 0.521. Tree models are very strong on table-like data, so our deep models must bring something extra.

Results

Transformer results on future payments

0.341

PR-AUC on the test period
(about 10× random
guessing)

27%

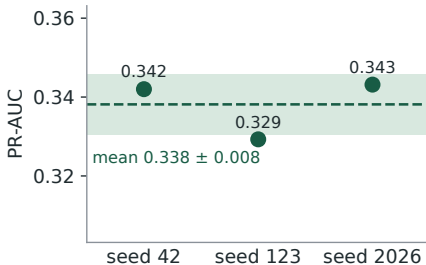
of fraud caught when only
1% of normal payments
are flagged

Fraud changes over time

During development the score was 0.44; on the newer test payments it fell to 0.34. That is why we always test on the future.

Is the gate stable?

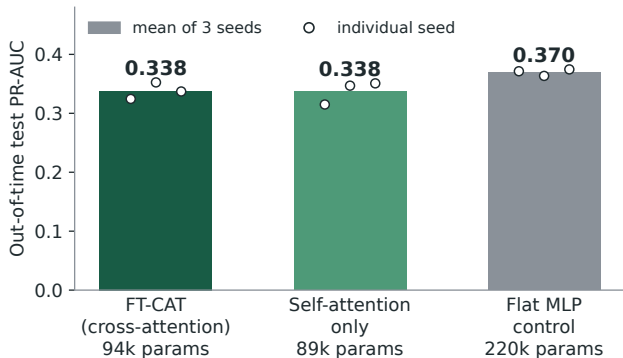
We trained it 3 times with different random starts:



Measured on the tuning period.

Results

Did attention help? An honest experiment



What we did

Trained three versions on the same data with the same settings, three times each.

What we found

The simple network scored best (0.370); both Transformers scored about 0.338.

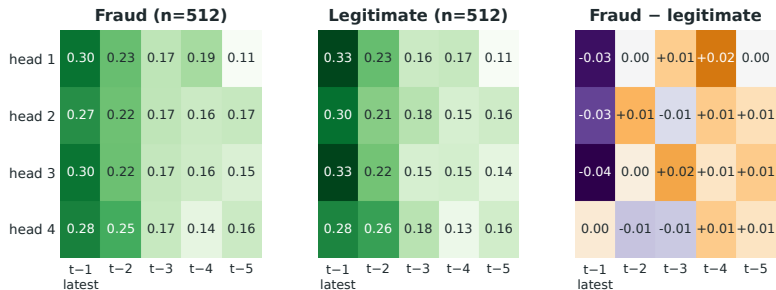
Fair warning

The simple network had more than twice as many parameters.

Fancier is not automatically better: complex models have to prove they are worth it.

Results

Looking inside: where does attention look?



How to read it

Each number: how much the model looks at one past payment (0.20 = equal attention).

What we see

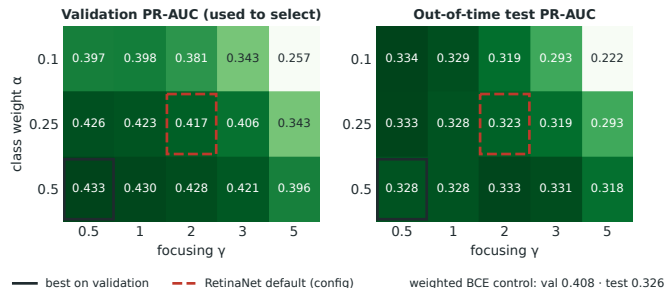
Mostly the latest payment, and almost the same for fraud and normal payments.

What it means

It is not finding fraud patterns in the history yet; its inputs need improving.

Results

Tuning the loss: the popular default was not best



What we tuned

Two focal-loss settings: focus on hard examples, and extra weight for fraud.

What we found

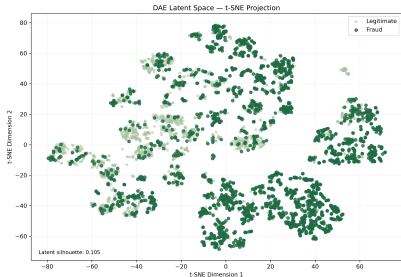
Gentle settings (0.5, 0.5) beat the popular default (2, 0.25); very strong focus hurt.

How big is the effect?

Small: within about 0.01 of a plain weighted loss (one run each).

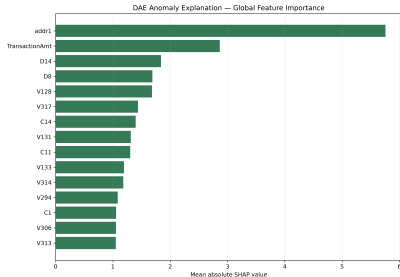
Results

What the autoencoder sees



A map of its 32 numbers

Each dot is a payment, flattened to 2D (t-SNE). Fraud (dark) and normal (light) overlap, so “unusual” does not always mean fraud: it helps, but cannot decide alone.



Which details look unusual?

SHAP measures how much each detail adds to the anomaly score. The address code (addr1) and the amount matter most.

Results

Fast enough for real-time payments

6.6 ms

to score one payment

2,746

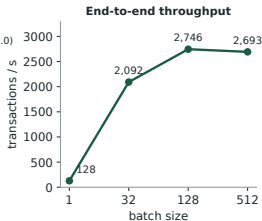
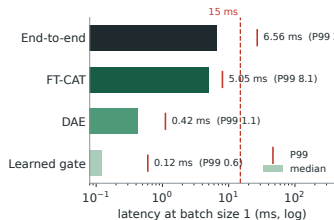
payments per second

CPU only

no graphics card needed

≈2 MB

autoencoder file size

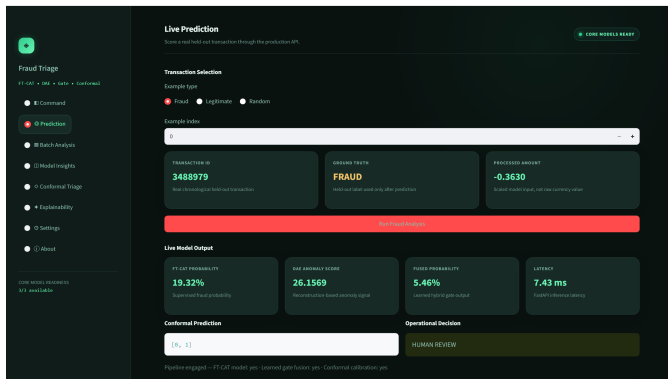


The Transformer takes most of the time. Typical: 6.6 ms; the slowest 1 in 100 take 27 ms or more (goal: 15 ms).

Measured on a single-CPU machine; model time only, without network overhead.

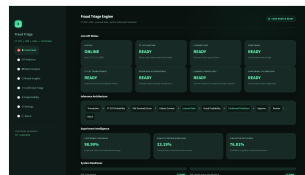
Results

The product: a dashboard for fraud analysts

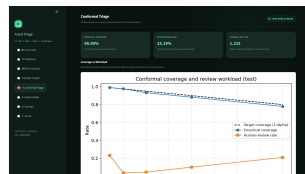


A real test payment is scored and sent to human review

LIVE DEMO →



System status



Confidence check results

Conclusion

Decisions, not just scores

76.8% of payments are approved automatically; humans review 23.2% and see 71% of all fraud.

Honestly tested

Two detectors combined by a gate, tested on future payments, and we show the results that did not go our way.

Ready to run

6.6 ms per payment on a normal CPU, with a web API and an analyst dashboard.

Next steps

- Separate safety targets for fraud and for normal payments
- Watch for changing fraud patterns and recalibrate regularly
- Explain the final decision, not just one part
- Find a safe setting that allows automatic blocking

Thank you

Questions?

APPROVE

REVIEW

BLOCK

github.com/sa6uhi/DL-final-project · report: report/report.pdf

Nadir Əskərov · Akşin Fətəliyev · Habil Hüseynov · Səbuhi Nəzərov